

Plantify

Android Plant Care & Identification Application

Technical Documentation • v1.0 • 2026

Author: Shon Yuddasin

1. Project Overview

Plantify is a free Android application that uses Artificial Intelligence to identify plants from photographs. Beyond identification, it provides a full plant-care ecosystem: a personal digital garden, configurable watering reminders, real-time weather data, and an AI-powered chatbot — all in one app.

Project Name	Plantify
Platform	Android (Java)
Submitted by	Shon Yudsin
Mentor	Maor Karkoukli
Institution	Atid Neve Nachum High School
Submission Date	17/05/2026
Core Problem	People own houseplants but don't know how to care for them or forget to water them
Solution	AI-powered identification + personal garden management + smart watering reminders
Competitors	Plantum, PlantNet, PictureThis — all require payment or restrict free features
Target Audience	Hobbyists, beginners, busy plant owners, botany enthusiasts

Key Innovations

- AI-based plant identification from a live camera shot or gallery image
- Integration of three external APIs in a single, free app (Plant.id, OpenWeatherMap, OpenAI GPT)
- Background watering notification system that works even when the app is closed
- Custom animated weather background rendered via a Canvas View subclass
- Daily Plant feature that surfaces a new plant of the day from a local dataset

2. Technology Stack

Technology	Category	Purpose
Java (Android SDK)	Core Language	All Activities, Fragments, Adapters, and Model classes are written in Java
Firebase Auth	Authentication	User registration and login via email/password and Google Sign-In
Firebase Realtime DB	Database	Cloud storage for users, plants, and reminders with real-time sync
SharedPreferences	Local Storage	Persists the user UID across sessions to skip login on relaunch
Plant.id API	AI / Vision	Sends a Base64-encoded image and receives species name, toxicity, watering frequency
OpenWeatherMap API	External API	Returns temperature, humidity and conditions for any searched location
OpenAI GPT-3.5-Turbo	AI Chat	Powers the in-app chatbot that answers plant-care questions in natural language
OkHttpClient	Networking	HTTP client used to send requests to all three external APIs
AlarmManager	Scheduling	Schedules watering alerts; fires even when the app is not running
BroadcastReceiver	Background	Wakes up on alarm, fires a Notification, reschedules or cleans up recurring reminders
CameraX / Gallery	Device I/O	Captures or imports photos; converts ImageProxy to Bitmap for API upload
ActivityResultContracts	Navigation	Modern contract pattern for gallery picker and Google Sign-In result handling
ObjectAnimator	UI	Page transitions, loading screens, and the animated weather background
XML Layouts	UI	Declarative UI via Activities, Fragments, BottomSheets, and custom dialogs

3. Architecture & Data Model

3.1 Application Architecture

Plantify follows the standard Android Activity / Fragment pattern with a single MainActivity that hosts a BottomNavigationView. Navigation between the main sections is handled by swapping Fragments within a NavHostFragment. Authentication screens (StartActivity, SignupActivity, SigninActivity) run as standalone Activities.

The project is organized into four logical layers:

- Activities — entry points: StartActivity, SignupActivity, SigninActivity, MainActivity
- Fragments — feature screens: Home, Garden, Camera, Picture, PlantDetail, AI, Water, Weather, DailyPlant, More, Settings, Problem, ProblemDetail
- Adapters — RecyclerView bridges: PlantAdapter, GardenStripAdapter, HomeReminderAdapter, MessageAdapter, WaterAdapter, SettingsAdapter, ProblemExpandableAdapter
- Models — pure data classes: User, Plant, Reminder, Message, DailyPlant, SettingItem, ProblemItem, ProblemGroup

3.2 Data Model

All persistent data lives in Firebase Realtime Database under the authenticated user's UID. The main objects and their key fields:

User	uid (Firebase), username, email, password
Plant	id, userId, name, scientificName, type, toxicity, watering, sun, soilType, photoUrl, notes, firebaseKey
Reminder	id, plantId, time, repeatType (once/days/interval), repeatInterval, dayOfWeek, date
Message	id, content, sender (bot user), timestamp
DailyPlant	name, shortDescription, longDescription, imageUrl, tags
ProblemGroup / ProblemItem	category header + list of individual problems with causes and solution steps

3.3 Key Relationships

- User ↔ Plant: Plant.userId = User.UID (each plant belongs to one user)
- Plant ↔ Reminder: Reminder.plantId = Plant.firebaseKey (each reminder targets one plant)
- Helper object ReminderWithPlants bundles a Reminder with its associated Plant name for the Water list view

4. What the User Can Do

4.1 Authentication

- Register with username, email, and a validated password (6-9 chars, uppercase, digit, special char)
- Sign in with email/password or via Google One-Tap
- App auto-skips login screens if a saved UID is detected in SharedPreferences
- Delete account or log out from the Settings screen

4.2 Plant Identification (Camera & Gallery)

- Open the camera screen, point at a plant, and capture a photo
- Alternatively, pick an existing image from the device gallery
- The app checks image brightness before upload (rejects photos that are too dark)
- Sends the image as Base64 to Plant.id API; receives species name, scientific name, toxicity level, and watering/sunlight recommendations
- Displays results in PictureFragment with an option to add the plant directly to the personal garden

4.3 Digital Garden

- Browse all personal plants in a grid/list view (GardenFragment)
- Add a plant manually or after identification — includes name, photo, notes, watering schedule
- Tap any plant to open PlantDetailFragment showing full care info, photo, and notes
- Edit or delete any plant using a contextual menu on long-press

4.4 Watering Reminders

- Attach one or more reminders to any plant
- Choose reminder type: one-time, specific weekday, or every N days (interval)
- Reminders fire as system Notifications even when the app is closed
- Recurring reminders auto-reschedule; one-time reminders self-delete from Firebase after firing
- The Water screen lists all upcoming reminders grouped by plant

4.5 AI Chatbot

- Ask plant-care questions in natural language via the AI Fragment
- Powered by OpenAI GPT-3.5-Turbo; full conversation history is maintained in-session
- Chat bubbles distinguish user messages from bot responses visually

4.6 Weather

- Search any city or region and receive current temperature, humidity, and conditions
- Animated weather background (custom View subclass) reacts to the weather type
- A BottomSheet lets the user select from preset weather regions

4.7 Daily Plant

- A featured plant is shown each day, loaded from a local JSON/storage dataset
- Includes name, short and long descriptions, image, and tags
- Accessible from the Home screen with a smooth animated transition

4.8 Common Problems

- Expandable list of plant problems organized by category (ProblemFragment)
- Each problem opens a detail view with causes and step-by-step solutions

4.9 Settings

- Toggle-based settings list (notifications, display preferences) stored in Firebase
- Delete account (with confirmation dialog) or log out

5. Key Code Samples

5.1 Session Check — StartActivity

On launch, the app immediately checks if a user is already logged in and skips the auth flow:

```
super.onCreate(savedInstanceState);
String uid = FirebaseHelper.getCurrentUserId(this);
if (uid != null) {
    startActivity(new Intent(this, MainActivity.class));
    finish();
}
```

5.2 Registration — SignupActivity

After validating inputs (username 3-12 chars, valid email, password with uppercase + digit + special char), the app creates the Firebase Auth account, saves the UID locally, and writes the User object to Realtime Database:

```
mAuth.createUserWithEmailAndPassword(email, password)
    .addOnCompleteListener(this, task -> {
        if (task.isSuccessful()) {
            String uid = mAuth.getUid();
            // Persist UID locally for auto-login
            getSharedPreferences("PlantifyPrefs", MODE_PRIVATE)
                .edit().putString("user_id", uid).apply();
            // Write user object to Firebase Realtime Database
            User user = new User(username, email, password);
            user.setUID(uid);
            FirebaseDatabase.getInstance().getReference()
                .child("User").child(uid).setValue(user);
            startActivity(new Intent(this, MainActivity.class));
        }
    });
```

5.3 Password Validation — Regex

Input validation uses regex patterns to enforce security rules:

```
// Username: 3-12 characters, letters/numbers/underscores only
if (!username.matches("[a-zA-Z0-9_]+$")) { ... }

// Password: 6-9 chars, must contain uppercase, digit, special char
if (!password.matches(".*[A-Z].*")) { ... }
if (!password.matches(".*[0-9].*")) { ... }
if (!password.matches(".*[!@#%^&*()].*")) { ... }
```

5.4 Image Brightness Check

Before sending a photo to the Plant.id API, the app samples every 10th pixel to compute average brightness and rejects dark images:

```
private boolean isTooDark(Bitmap bitmap) {
    int width = bitmap.getWidth();
    int height = bitmap.getHeight();
    long totalBrightness = 0; int pixelCount = 0;
    for (int x = 0; x < width; x += 10) {
        for (int y = 0; y < height; y += 10) {
            int pixel = bitmap.getPixel(x, y);
            int r = (pixel >> 16) & 0xff;
            int g = (pixel >> 8) & 0xff;
            int b = pixel & 0xff;
            totalBrightness += (r + g + b) / 3;
            pixelCount++;
        }
    }
    return (int)(totalBrightness / pixelCount) < 80;
}
```

5.5 Background Thread for API Calls

Network calls run on a background thread to prevent the UI from freezing; results are posted back to the main thread:

```
new Thread(() -> {
    // Send image to Plant.id and parse JSON response
    String result = PlantIdentifier.identify(base64Image);
    requireActivity().runOnUiThread(() -> {
        // Update UI with plant data
        tvPlantName.setText(result);
    });
}).start();
```

5.6 BroadcastReceiver — Watering Reminder

When AlarmManager fires, ReminderReceiver decides whether to reschedule (recurring) or delete (one-time) the reminder:

```
// Step 1: Fire the system notification
fireNotification(context, plantName, plantId);

// Step 2: Handle recurrence
boolean isOneTime = "once".equals(repeatType) || repeatType == null;
if (isOneTime) {
    deleteReminderFromFirebase(context, reminderId);
} else if ("days".equals(repeatType) && dayOfWeek > 0) {
    // Reschedule same weekday next week
    AlarmScheduler.rescheduleDayAlarm(
        context, reminder, plantName, dayOfWeek, hour, minute);
} else if ("interval".equals(repeatType)) {
    // Reschedule after N days
    AlarmScheduler.rescheduleInterval(
        context, reminder, plantName, hour, minute);
}
```

5.7 Animated Weather Background

The weather screen uses a custom View subclass that draws and animates its own canvas:


```
public class AnimatedWeatherBackground extends View {  
    // Override onDraw() to paint animated clouds, rain drops,  
    // sun rays, or snowflakes depending on weather condition.  
    // Uses ObjectAnimator / invalidate() loop for smooth animation.  
}
```

6. Core Data Flows

6.1 Plant Identification Flow

- User opens CameraFragment and captures a photo
- CameraX delivers an ImageProxy; app converts it to Bitmap via imageToBitmap()
- isTooDark() checks average pixel brightness; blocks upload if < 80
- Bitmap is compressed and Base64-encoded on a background Thread
- OkHttpClient sends POST to Plant.id API with image payload and API key
- JSON response is parsed: species name, scientific name, toxicity, watering, sunlight
- runOnUiThread() updates PictureFragment with results
- User can tap 'Add to Garden' to persist the plant to Firebase

6.2 Watering Reminder Flow

- User sets a reminder in ReminderBottomSheet (time, repeat type)
- Reminder object is saved to Firebase under the plant's node
- AlarmScheduler calls AlarmManager.setExactAndAllowWhileIdle() for the next trigger time
- At trigger time, ReminderReceiver.onReceive() fires (even if app is closed)
- Notification is pushed to the device status bar
- Receiver reschedules recurring alarms or deletes one-time ones from Firebase

6.3 AI Chat Flow

- User types a message in AiFragment
- Message is appended to the in-memory conversation history list
- Full history is sent to OpenAI GPT-3.5-Turbo API via OkHttpClient
- Response is parsed and displayed as a bot bubble in the RecyclerView

7. Permissions & Constraints

CAMERA	Required to capture plant photos in CameraFragment
READ_EXTERNAL_STORAGE	Required to pick images from the device gallery
INTERNET	Required for all API calls and Firebase sync
RECEIVE_BOOT_COMPLETED	Required to reschedule alarms after device reboot (BootReceiver)
POST_NOTIFICATIONS	Required (Android 13+) to display watering reminders
SCHEDULE_EXACT_ALARM	Required for precise watering reminder timing

Known Limitations

- Chat history is in-memory only — it resets when the app is closed
- Plant.id identification accuracy depends on photo quality and lighting
- Weather data requires an active internet connection
- Exact alarms may be delayed on devices with aggressive battery optimization

8. Project Structure Summary

Activities (4)	StartActivity, SignupActivity, SigninActivity, MainActivity
Fragments (13)	Home, Garden, Camera, Picture, PlantDetail, AI, Water, Weather, DailyPlant, More, Settings, Problem, ProblemDetail
Adapters (7)	PlantAdapter, GardenStripAdapter, HomeReminderAdapter, MessageAdapter, WaterAdapter, SettingsAdapter, ProblemExpandableAdapter
Models (8)	User, Plant, Reminder, Message, DailyPlant, SettingItem, ProblemItem, ProblemGroup
Helpers	FirebaseHelper (DB/Auth abstraction), AlarmScheduler, PlantIdentifier
Receivers	ReminderReceiver (alarm trigger), BootReceiver (reschedule after reboot)
Custom Views	AnimatedWeatherBackground (Canvas animation)
External APIs (3)	Plant.id (identification), OpenWeatherMap (weather), OpenAI GPT-3.5-Turbo (chat)
Firebase Services (2)	Firebase Authentication, Firebase Realtime Database
Networking	OkHttpClient for all HTTP communication

The app was built and tested on Android and references roughly 469 pages of full documentation covering all screens, class diagrams, UML, XML layouts, and complete Java source code.